



Twelve

Python Tips for Beginners

In a previous article published in the March 2021 edition of *Open Source For You* ('Python: A Few Useful Tips for Beginners'), the author of this article had written some simple but useful tips for those beginning to learn the Python language. He has now compiled twelve more tips that are interesting and useful for Python programmers. These tips will help write concise and easy-to-read code.

As per both the TIOBE index and the PYPL (Popularity of Programming Language) site, as on September 2022, Python is the most popular programming language and has retained its popularity for almost three years now. This is because of its ease of programming and the availability of a very large number of libraries covering a large number of areas, right from business and finance to scientific computing and AI. It is also easy to learn compared to other languages like C/C++ and Java.

So let's begin with the Python tips that will help you write concise code easily.

Use of underscore to separate digits of long numbers

It is difficult to count the number of digits if a number, be it an integer or floating-point number, is very long. Python allows the use of underscores (_) as visual separators for grouping digits to make it easier to read the number. Hence, the number 1000000000 can be written as 1_000_000_000.

```
>>> num = 1_000_000_000
>>> num
1000000000
```

List unpacking and partial assignment

If we need to assign some of the elements of a list to

particular variables and the remaining values to some other variable, the '*' notation for unpacking can be used in Python.

```
>>> numlist1 = [ 1,2,3,4,5 ]
>>> num1, *numlist2 = numlist1
>>> print ( "num1 = ", num1 )
num1 = 1
>>> print ( " numlist2 = ", numlist2 )
numlist2 = [ 2,3,4,5 ]
```

Unpacking is used in the assignment statement below to assign values from the list to three variables simultaneously, and the '*' is used to assign all the remaining values.

```
>>> numlist1 = [ 1,2,3,4,5 ]
>>> num1, *numlist2, num3 = numlist1
>>> print ("num1 = ", num1)
num1 = 1
>>> print ( "numlist2 = ", numlist2 )
numlist2 = [ 2, 3, 4 ]
>>> print ( "num3 = ", num3 )
num3 = 5
```

Use of underscore in unpacking

The underscore (_) can also be used to ignore specific values while unpacking. The value that has to be ignored is assigned